Bahasa Pemrograman Tingkat Rendah (MWCA010)

Program Studi Teknik Informatika ▪ Fakultas Teknologi Informasi ▪ Universitas Bale Bandung

Interupsi BIOS/DOS dan Operasi I/O

Pertemuan 13

Dosen Pengampu

Cecep Suwanda, S.Si., M.Kom.

Semester Ganjil

Capaian dan Tujuan Pembelajaran

Sub-CPMK 2.4

Menjelaskan mekanisme interupsi dan layanan BIOS/DOS (`INT 10h`, `INT 21h`) sebagai jembatan antara instruksi mesin dan perangkat keras serta operasi masukan/keluaran.

CPMK yang didukung: CPMK02 ▪ **CPL yang didukung: CPL02**

Setelah pertemuan ini, mahasiswa mampu:

1. menjelaskan alasan diperlukannya interupsi dan membedakan tiga jenisnya;
2. menghitung alamat vektor interupsi ke-*n* dan menyebutkan vektor-vektor penting beserta kegunaannya;
3. menelusuri penyimpanan `FLAGS`, `CS`, dan `IP` pada stack serta pengambilannya kembali oleh `IRET`;
4. menggunakan layanan BIOS `INT 10h`, `INT 16h`, `INT 17h` serta layanan DOS `INT 21h`;
5. memasang *interrupt handler* sederhana dengan mengganti isi sebuah vektor, lalu memulihkan vektor tersebut.

Peta Materi

1. **Konsep Interupsi dan Tabel Vektor**
   polling dan interupsi; tiga jenis interupsi; tabel vektor `0000h`–`03FFh`; alamat vektor ke-*n*; mekanisme `INT` dan `IRET`; `TF`

2. **Interupsi BIOS: Video, Keyboard, dan Printer**
   kedudukan BIOS; `INT 10h` (fungsi `09h`, `13h`, `02h`); byte atribut warna; `INT 16h`; `INT 17h`

3. **Interupsi DOS dan Penangan Buatan Sendiri**
   fungsi `INT 21h` (`0Ah`, `09h`, `02h`, `25h`, `35h`, `4Ch`); penyangga fungsi `0Ah`; `INT 20h`; pemasangan dan pemulihan vektor; interupsi berantai

Praktik pada pertemuan ini

Operasi masukan/keluaran dengan `INT 21h` dan penelusuran isi stack sebelum serta sesudah pemanggilan `INT` pada `input-nama.asm` dan `warna-layar.asm`.

BAGIAN

Konsep Interupsi dan Tabel Vektor

Mengapa Interupsi Diperlukan

- Prosesor berjalan jauh lebih cepat daripada perangkat di sekitarnya. Bila program terus-menerus memeriksa status keyboard, sebagian besar waktunya terbuang untuk pengulangan yang hampir selalu sia-sia — cara ini disebut *polling*.
- **Interupsi** adalah mekanisme yang memaksa prosesor menghentikan sementara alur program, memindahkan kendali ke rutin khusus yang disebut **penangan interupsi** (*interrupt handler*), lalu melanjutkan kembali program semula tepat pada instruksi berikutnya.
- Interupsi juga menjadi sarana baku untuk meminta layanan sistem: program cukup memanggil `INT 10h` tanpa menulis sendiri rutin pengendali video.

Kotak hitam yang dibuka

Sejak Bab III, `INT 21h` dipakai sebagai kotak hitam. Pertanyaannya kini bukan lagi "bagaimana memakai" layanan, melainkan "apa yang sesungguhnya terjadi" ketika `INT` dijalankan.

Sumber: Bab XIV, Mengapa Interupsi Diperlukan

Tiga Jenis Interupsi

Jenis	Sumber	Contoh	Sifat
Perangkat keras	Kaki `INTR` atau `NMI`	Timer 18,2 kali per detik (`08h`); penekanan tombol (`09h`)	Terjadi kapan saja, tidak direncanakan program
Perangkat lunak	Instruksi `INT n`	`INT 21h` fungsi `09h` untuk mencetak string	Disengaja, hasilnya dapat diperkirakan
Pengecualian	Prosesor, akibat kesalahan eksekusi	Pembagian dengan nol (`00h`); limpahan `INTO` (`04h`)	Menandakan kesalahan yang harus ditangani

Interupsi melalui `INTR` bersifat *maskable*: dapat ditahan bila `IF` bernilai nol. Interupsi `NMI` tidak dapat ditolak dan dipakai untuk kejadian gawat.

Sumber: Bab XIV, Tiga Jenis Interupsi

Tabel Vektor Interupsi

vektor	alamat	isi 4 byte		keterangan
00h	0000h	offset	segmen	pembagian dengan nol
09h	0024h	offset	segmen	keyboard
10h	0040h	offset	segmen	layanan video BIOS
21h	0084h	offset	segmen	layanan DOS

256 kemungkinan interupsi diberi nomor 00h sampai FFh. Tabelnya menempati 1.024 byte pertama memori — alamat 0000h sampai 03FFh pada segmen nol — sehingga mampu menampung $1.024 \div 4 = 256$ vektor.

Setiap entri berisi 2 byte **offset** penangan (dimuat ke IP) diikuti 2 byte **segmen** (dimuat ke CS); bagian bernilai lebih rendah diletakkan pada alamat yang lebih rendah. Entri berisi *alamat* penangan, bukan penangannya sendiri.

Sumber: Bab XIV, Tabel Vektor Interupsi

Menghitung Alamat Vektor ke-n

Karena setiap vektor berukuran tetap, alamat awal vektor bernomor n dihitung dengan rumus berikut.

$$\text{alamat vektor}_n = n \times 4$$

Hasilnya adalah *offset di dalam segmen nol*.

Vektor	**Perhitungan**	**Alamat entri**
`09h` keyboard	$9 \times 4 = 36$	`24h`
`10h` video BIOS	$16 \times 4 = 64$	`40h`
`21h` layanan DOS	$33 \times 4 = 132$	`84h`

Offset inilah yang dipakai program untuk mengganti atau membaca entri vektor, misalnya melalui fungsi `25h` dan `35h` layanan DOS.

Sumber: Bab XIV, Menghitung Alamat Vektor ke-n

Vektor Interupsi yang Penting

Vektor	Alamat	Pemicu	Kegunaan
00h	0000h	Pengecualian prosesor	Pembagian dengan nol, dipicu oleh DIV
01h	0004h	Pengecualian prosesor	Eksekusi satu langkah, dipicu saat TF = 1
08h	0020h	Perangkat keras	Timer sistem, 18,2 kali per detik
09h	0024h	Perangkat keras	Penekanan dan pelepasan tombol
10h	0040h	Perangkat lunak	Layanan video BIOS: mode, kursor, karakter
16h	0058h	Perangkat lunak	Layanan keyboard BIOS: baca dan periksa tombol
17h	005Ch	Perangkat lunak	Layanan printer BIOS
1Ah	0068h	Perangkat lunak	Waktu dan tanggal dari jam perangkat keras
1Ch	0070h	Perangkat lunak	Timer *tick* bagi pengguna, dipanggil dari 08h
21h	0084h	Perangkat lunak	Layanan DOS: masukan, keluaran, berkas, terminasi

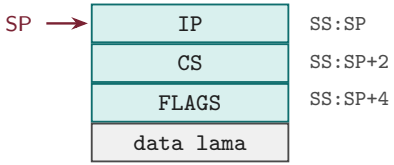
Sumber: Bab XIV, Menghitung Alamat Vektor ke-n

Mekanisme INT dan IRET

`INT n` bekerja seperti pemanggilan prosedur jarak jauh, dengan tambahan penyimpanan *flag*:

1. `FLAGS` disimpan ke stack;
2. `CS` disimpan ke stack;
3. `IP` — sudah menunjuk instruksi sesudah `INT n` — disimpan ke stack;
4. `IF` dan `TF` dikosongkan;
5. alamat penangan dibaca dari entri vektor pada alamat $n \times 4$: dua byte pertama ke `IP`, dua byte berikutnya ke `CS`.

Ketiga penyimpanan itu menurunkan `SP` sebanyak 6 byte.



Penangan wajib diakhiri IRET, bukan RET

`IRET` mengambil `IP`, `CS`, lalu `FLAGS` — kebalikan urutan penyimpanan; setara `RET` ditambah `POPF`. Bila `RET` yang dipakai, `FLAGS` terambil sebagai alamat kembali sehingga prosesor melompat ke alamat acak.

Sumber: Bab XIV, Mekanisme INT dan IRET

Flag TF dan Eksekusi Satu Langkah

- Bit ke-8 register FLAGS adalah **trap flag** TF. Bila bit ini bernilai satu, prosesor menjalankan satu instruksi lalu segera membangkitkan interupsi vektor 01h.
- Mekanisme inilah yang dipakai *debugger* untuk menyediakan tombol Single Step pada Emu8086 atau perintah T pada DEBUG.
- Peran TF menjelaskan langkah keempat mekanisme INT: bila flag itu tetap bernilai satu, penangan akan memicu vektor 01h setiap kali satu instruksinya selesai sehingga prosesor terjebak.

T dan P pada DEBUG

Perintah T (*trace*) menjalankan satu instruksi dan *masuk* ke rutin penangan, sedangkan P (*proceed*) menjalankannya sampai selesai tanpa masuk ke penangan.

Sumber: Bab XIV, Flag TF dan Eksekusi Satu Langkah

Rangkuman Bagian 1

- Interupsi lebih hemat daripada *polling*: prosesor dipanggil hanya ketika ada kejadian, lalu melanjutkan program tepat pada instruksi berikutnya. Interupsi juga menjadi sarana baku meminta layanan sistem.
- Berdasarkan sumbernya ada tiga jenis: perangkat keras (`INTR` *maskable* dan `NMI`), perangkat lunak (`INT n`), dan pengecualian yang dibangkitkan prosesor.
- Tabel vektor menempati `0000h`–`03FFh` pada segmen nol; setiap vektor 4 byte (offset lalu segmen), sehingga alamat entri vektor ke-n adalah $n \times 4$.
- `INT n` menyimpan `FLAGS`, `CS`, lalu `IP` (SP turun 6 byte), mengosongkan `IF` dan `TF`, lalu memuat `CS:IP` dari vektor. `IRET` mengembalikan ketiganya secara terbalik.
- `TF` adalah dasar eksekusi satu langkah pada *debugger*: satu instruksi, lalu interupsi vektor `01h`.

BAGIAN

Interupsi BIOS: Video, Keyboard, dan Printer

Kedudukan BIOS

- **BIOS** (*Basic Input/Output System*) adalah kumpulan rutin yang tersimpan pada cip memori baca-saja dan dipetakan ke alamat `F000h` pada ruang memori.
- Rutin ini berbicara langsung dengan pengendali video, pengendali keyboard, dan port printer, lalu menyajikan antarmuka yang seragam bagi program di atasnya.
- Karena perbedaan perangkat keras disembunyikan di sini, program tidak perlu mengenal nomor port perangkat.

Konvensi seluruh layanan BIOS

`AH` berisi nomor fungsi, register lain berisi argumen, dan hasil dikembalikan pada register.

Sumber: Bab XIV, Kedudukan BIOS

INT 10h: Layanan Video (1/2)

Interupsi 10h melayani keperluan tampilan. Fungsi yang paling sering dipakai pada praktikum:

Fungsi	Register masukan	Register keluaran	Keterangan
00h	AL = nomor mode video	—	Menetapkan mode; 03h berarti teks 80 × 25 berwarna, dan layar dibersihkan
02h	BH = halaman; DH = baris; DL = kolom	—	Memindahkan posisi kursor tanpa mencetak apa pun
03h	BH = halaman	DH, DL = posisi kursor; CX = bentuk kursor	Membaca posisi kursor, berguna untuk menyimpan lalu memulihkan posisi
06h	AL = jumlah baris gulung; BH = atribut; CH, CL, DH, DL = batas jendela	—	Menggulung jendela ke atas; AL = 00h menggulung seluruh jendela

Sumber: Bab XIV, INT 10h: Layanan Video

INT 10h: Layanan Video (2/2)

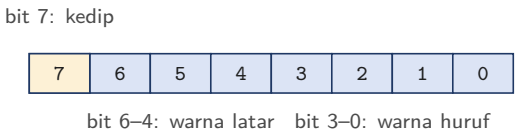
Fungsi	Register masukan	Register keluaran	Keterangan
09h	AL = karakter; BH = halaman; BL = atribut; CX = jumlah penulisan	—	Menulis karakter berwarna berulang; kursor *tidak* bergeser
0Ah	AL = karakter; BH = halaman; CX = jumlah penulisan	—	Menulis karakter memakai atribut yang sudah ada di layar; kursor juga tidak bergeser
13h	AL = mode tulis; BH = halaman; BL = atribut; CX = panjang string; DH, DL = posisi; ES:BP = alamat string	—	Menulis seluruh string sekaligus; mode AL = 01h memindahkan kursor ke akhir string

Fungsi 09h menulis satu karakter yang dapat diulang sebanyak nilai CX tanpa menggeser kursor, sehingga tepat untuk membuat blok warna. Fungsi 13h mencetak seluruh string pada ES:BP, sehingga DS harus disalin ke ES lebih dahulu.

Sumber: Bab XIV, INT 10h: Layanan Video

Byte Atribut Warna

Warna pada mode teks ditentukan satu byte **atribut** dengan susunan bit yang tetap.



`BL` = `0Eh` berarti `00001110b`: bit 3–0 bernilai `Eh` (kuning terang) dan bit 6–4 bernilai `0h` (latar hitam).

Atribut yang sering dipakai

`07h` abu-abu terang pada latar hitam (bawaan DOS), `0Eh` kuning, `1Fh` putih pada latar biru, `4Eh` kuning pada latar merah.

Pemakaian `INT 10h` lebih aman daripada menulis langsung ke memori video pada alamat `B800h`, karena BIOS sudah mengurus alamat karakter dan atributnya.
Sumber: Bab XIV, Byte Atribut Warna

INT 16h: Layanan Keyboard

Interupsi 16h menyediakan tiga fungsi utama.

1. **Fungsi 00h — baca tombol.** Prosesor menunggu sampai ada tombol ditekan, lalu mengembalikan AH = *scan code* dan AL = kode ASCII. Untuk tombol panah, kode ASCII bernilai nol sehingga AH yang harus diperiksa.
2. **Fungsi 01h — periksa status tombol.** Fungsi ini tidak menunggu. Bila ada tombol pada penyangga keyboard, ZF bernilai nol dan AX berisi tombol tersebut tanpa mengeluarkannya.
3. **Fungsi 02h — status tombol pengubah.** Mengembalikan keadaan Shift, Ctrl, Alt, dan Caps Lock pada AL.

BIOS tidak melakukan *echo*

Masukan DOS melakukan *echo* ke layar dan menghormati tombol Ctrl+C, sedangkan masukan BIOS tidak melakukan keduanya. Program yang memakai BIOS bertanggung jawab mencetak sendiri karakter yang dibaca.

Sumber: Bab XIV, INT 16h: Layanan Keyboard

INT 17h: Layanan Printer

- Interupsi 17h mengirim satu byte ke printer melalui *port* paralel.
- Fungsi 00h mencetak karakter pada AL, fungsi 01h menginisialisasi printer, dan fungsi 02h membaca statusnya.

Bit status pada AH

- bit 1 — waktu tunggu habis
- bit 3 — kesalahan masukan/keluaran
- bit 5 — kertas habis
- bit 7 — printer sibuk

Sumber: Bab XIV, INT 17h: Layanan Printer

Rangkuman Bagian 2

- BIOS adalah kumpulan rutin pada cip ROM yang dipetakan ke `F000h` dan menyembunyikan perbedaan perangkat keras; seluruh layanannya memakai `AH` sebagai nomor fungsi.
- `INT 10h` melayani video: `00h` menetapkan mode, `02h` memindahkan kursor, `03h` membacanya, `09h` menulis karakter berulang tanpa menggeser kursor, dan `13h` menulis seluruh string pada `ES:BP`.
- Warna ditentukan byte atribut: bit 3–0 huruf, bit 6–4 latar, bit 7 kedip; `0Eh` kuning pada latar hitam.
- `INT 16h` melayani keyboard (fungsi `00h`, `01h`, `02h`) tanpa *echo* dan tanpa menghormati `Ctrl+C`.
- `INT 17h` mengirim satu byte ke printer melalui *port* paralel dan mengembalikan bit status pada `AH`.

BAGIAN

Interupsi DOS dan Penangan Buatan Sendiri

Dua Tingkat Layanan

- BIOS menyediakan akses ke perangkat keras, tetapi belum mengenal berkas, direktori, atau konsep "program yang sedang berjalan".
- Semua itu adalah tugas **DOS** (*Disk Operating System*) yang dibangun di atas BIOS melalui interupsi `21h`.
- **Penting:** BIOS menggerakkan perangkat, DOS mengatur pemakaian sistem secara keseluruhan.

Konvensi pemanggilan `INT 21h`

Sama dengan konvensi BIOS: `AH` berisi nomor fungsi dan register lain berisi argumen.

Sumber: Bab XIV, Dua Tingkat Layanan

Fungsi INT 21h yang Penting (1/2)

Fungsi	Register masukan	Register keluaran	Keterangan
01h	AH = 01h	AL = karakter	Membaca satu karakter dan menampilkannya (*echo*); menunggu sampai ada tombol
02h	AH = 02h; DL = karakter	—	Mencetak satu karakter pada layar
09h	AH = 09h; DS:DX = alamat string	—	Mencetak string yang diakhiri tanda '$'; tanda itu sendiri tidak dicetak

Pasangan DS:DX adalah bentuk baku alamat jauh pada 8086: DS menyimpan nomor segmen dan DX menyimpan offset di dalamnya. Karena itu program .EXE yang memakai fungsi 09h wajib menginisialisasi DS.

Sumber: Bab XIV, Fungsi INT 21h yang Penting

Fungsi INT 21h yang Penting (2/2)

Fungsi	Register masukan	Register keluaran	Keterangan
0Ah	AH = 0Ah; DS:DX = alamat penyangga	Penyangga terisi	Membaca string yang diketik pengguna sampai tombol Enter; karakter ditampilkan (*echo*)
25h	AH = 25h; AL = nomor vektor; DS:DX = alamat penangan	—	Mengganti isi entri vektor dengan alamat penangan baru
35h	AH = 35h; AL = nomor vektor	ES:BX = alamat penangan	Membaca isi entri vektor yang sedang berlaku
4Ch	AH = 4Ch; AL = kode keluar	—	Mengakhiri program, menutup berkas, dan mengembalikan kendali ke DOS

Enam fungsi di atas — bersama tiga fungsi bagian sebelumnya — adalah seluruh layanan DOS yang dipakai sepanjang bab ini.

Sumber: Bab XIV, Fungsi INT 21h yang Penting

Penyangga Fungsi 0Ah

Fungsi 0Ah berbeda dari fungsi lain karena pengguna juga menyediakan penyangga (*buffer*) yang tata letaknya ditentukan DOS.

```
.DATA
    nama    DB 20           ; byte 0: kapasitas maksimum,
                            ;   termasuk tombol Enter
            DB 0            ; byte 1: jumlah karakter yang diketik,
                            ;   diisi oleh DOS
            DB 20 DUP(0)    ; byte 2 dan seterusnya: isi yang diketik
```

Byte pertama harus diisi *sebelum* pemanggilan, byte kedua diisi DOS dengan jumlah karakter yang benar-benar diketik. DOS juga menambahkan 0Dh, tetapi *tidak* menambahkan tanda '$'.

Tanda '$' harus disisipkan program

Karena fungsi 09h berhenti hanya ketika menemukan '$', tanda itu dituliskan pada posisi penyangga + 2 + jumlah karakter. Bila tidak, fungsi 09h terus mencetak isi memori sesudah penyangga sampai menemukan byte 24h — layar dipenuhi karakter sampah.

Sumber: Bab XIV, Penyangga Fungsi 0Ah

INT 20h dan INT 21h Fungsi 4Ch

Aspek	INT 20h	INT 21h fungsi 4Ch
Bentuk program	Cocok untuk .COM	Dapat dipakai .COM maupun .EXE
Prasyarat	CS harus menunjuk ke segmen awal program	Tidak ada prasyarat khusus
Kode keluar	Tidak dapat disertakan	Diberikan pada AL, dapat dibaca program pemanggil
Pembersihan oleh DOS	Terbatas	Menutup berkas terbuka dan memulihkan vektor Ctrl+C
Pemakaian pada praktikum	Jarang, hanya pada contoh .COM	Bentuk baku program .EXE

Fungsi 4Ch lebih umum dan lebih aman sehingga menjadi pilihan baku; INT 20h tetap perlu dikenal karena muncul pada contoh program lama.

Sumber: Bab XIV, INT 20h dan INT 21h Fungsi 4C

Memasang Penangan Interupsi Sendiri

Mengganti isi sebuah entri vektor berarti mengganti seluruh perilaku interupsi tersebut. Contoh memakai vektor 60h yang tidak dipakai perangkat keras mana pun.

```
MOV AH, 35h                 ; baca vektor lama
MOV AL, 60h
INT 21h                     ; ES:BX = penangan lama
PUSH ES
PUSH BX                     ; simpan di stack
PUSH DS                     ; simpan DS program
MOV DX, OFFSET handler60    ; DX = offset baru
MOV AX, SEG handler60       ; AX = segmen baru
MOV DS, AX                  ; DS:DX = penangan baru
MOV AH, 25h                 ; pasang vektor
MOV AL, 60h
INT 21h
POP DS
INT 60h                     ; uji penangan baru
```

```
handler60 PROC FAR
    PUSH AX                 ; simpan register
    PUSH DX
    MOV AH, 09h             ; cetak pesan
    LEA DX, pesan
    INT 21h
    POP DX                  ; pulihkan register
    POP AX
    IRET                    ; pop IP, CS, FLAGS
handler60 ENDP
```

Lima langkah: simpan penangan lama (fungsi 35h), siapkan alamat baru pada DS:DX, pasang (fungsi 25h), uji dengan INT n, lalu pulihkan penangan lama. Penangan adalah prosedur jauh yang diakhiri IRET; karena dipanggil lewat interupsi, register yang dipakainya harus disimpan dan dipulihkan sendiri.

Memulihkan Vektor dan Risikonya

Memulihkan vektor lama

```
POP DX          ; offset penangan
    lama
POP DS          ; segmen penangan
    lama
MOV AH, 25h     ; pasang kembali
MOV AL, 60h
INT 21h
```

Interupsi berantai

Penangan yang menggantikan interupsi perangkat keras melanjutkan kendali ke penangan lama dengan *melompat* ke alamatnya, bukan dengan `IRET`.

Tiga kesalahan yang paling sering terjadi

- penangan tidak diakhiri `IRET`, sehingga prosesor melompat ke alamat acak;
- vektor lama tidak dipulihkan, sehingga program berikutnya menerima penangan yang sudah tidak ada;
- penangan tidak menyimpan register yang dipakainya.

Vektor `08h` dan `09h` sebaiknya tidak dijadikan percobaan pertama.

Sumber: Bab XIV, Memasang Penangan Interupsi Sendiri

Rangkuman Bagian 3

- BIOS menggerakkan perangkat, DOS mengatur pemakaian sistem; keduanya memakai `AH` sebagai nomor fungsi.
- Fungsi `INT 21h` yang penting: masukan (`01h`, `0Ah`), keluaran (`02h`, `09h`), pengelolaan vektor (`25h`, `35h`), dan terminasi (`4Ch`).
- Penyangga fungsi `0Ah`: byte 0 kapasitas, byte 1 jumlah karakter (diisi DOS), byte 2 dan seterusnya isi yang diketik. Tanda `'$'` harus disisipkan program bila isi penyangga akan dicetak dengan fungsi `09h`.
- Fungsi `4Ch` lebih umum dan lebih aman daripada `INT 20h`, sehingga menjadi pilihan baku program `.EXE`.
- Penangan baru wajib menyimpan register yang dipakainya dan diakhiri `IRET`; vektor lama wajib dipulihkan sesudah pengujian.

Contoh 1: Menyapa Pengguna dengan INT 21h

Meminta nama pengguna dengan fungsi 0Ah, menyisipkan tanda '$', lalu menyapa pengguna dengan nama tersebut.

```
.MODEL SMALL
.STACK 100h
.DATA
    tanya  DB 'Siapa nama Anda? $'
    sapaan DB 0Dh,0Ah,'Halo, $'
    nama   DB 20          ; kapasitas
           DB 0           ; jumlah terisi
           DB 20 DUP(0)   ; isi nama
    penutup DB '! Selamat belajar assembly.'
      ,0Dh,0Ah,'$'
.CODE
START:
    MOV AX, @DATA
    MOV DS, AX            ; DS -> .DATA
    MOV AH, 09h
    LEA DX, tanya
    INT 21h               ; cetak pertanyaan
```

```
    MOV AH, 0Ah           ; baca nama
    LEA DX, nama
    INT 21h               ; DOS isi byte 1 dan 2..
    LEA SI, nama
    MOV BL, [SI+1]        ; jumlah karakter
    MOV BH, 0             ; BX = jumlah karakter
    ADD SI, 2             ; SI -> nama pertama
    ADD SI, BX            ; SI -> sesudah terakhir
    MOV BYTE PTR [SI], '$'
    MOV AH, 09h
    LEA DX, sapaan
    INT 21h               ; cetak "Halo, "
    LEA DX, nama+2        ; cetak nama pengguna
    INT 21h
    LEA DX, penutup
    INT 21h
    MOV AH, 4Ch
    INT 21h               ; terminasi program
```

Keluaran: `Siapa nama Anda?` `Andi`, lalu `Halo, Andi! Selamat belajar assembly.` Fungsi 0Ah sudah menampilkan (*echo*) karakter yang diketik. Sumber: `input-nama.asm`.

Contoh 2: Teks Berwarna dengan `INT 10h`

Judul berwarna kuning dan garis merah dengan layanan video BIOS; tidak satu pun layanan DOS dipakai untuk mencetak.

```
.MODEL SMALL
.STACK 100h
.DATA
    judul   DB 'Bahasa Assembly - '
            DB 'Interupsi BIOS/DOS'
    panjang EQU $ - judul    ; 36
.CODE
START:
    MOV AX, @DATA
    MOV DS, AX
    MOV ES, AX          ; ES = DS
    MOV AH, 00h         ; set mode
    MOV AL, 03h         ; teks 80x25
    INT 10h
    MOV AH, 13h         ; tulis string
    MOV AL, 01h         ; kursor maju
    MOV BL, 0Eh         ; kuning
    MOV CX, panjang
    MOV DH, 05h         ; baris 5
    MOV DL, 0Ah         ; kolom 10
    LEA BP, judul
    INT 10h             ; ES:BP
    MOV AH, 02h         ; pindah kursor
    MOV DH, 07h         ; baris 7
    MOV DL, 0Ah         ; kolom 10
    INT 10h
    MOV AH, 09h         ; garis merah
    MOV AL, '='
    MOV BL, 0Ch         ; merah terang
    MOV CX, panjang
    INT 10h             ; kursor tetap
    MOV AH, 4Ch         ; terminasi
    MOV AL, 00h
    INT 21h
```

Karena yang memindahkan kursor adalah BIOS, DOS tidak mengetahui posisi kursor yang sesungguhnya; kedua cara pencetakan sebaiknya tidak dicampur tanpa mengatur ulang kursor. Sumber: `warna-layar.asm`.

Praktikum: Masukan/Keluaran dan Penelusuran Stack

Judul praktikum

Masukan/keluaran dengan `INT 21h` dan penelusuran stack pada pemanggilan `INT` ▪ **Sub-CPMK:** 2.4 ▪ 100 menit ▪ individu ▪ **Tugas 7 dikumpulkan**

Alat dan bahan

- Emu8086, GUI Turbo Assembler, atau DOSBox
- Berkas `input-nama.asm` dan `warna-layar.asm`
- Jurnal untuk mencatat register dan isi stack

Langkah kerja

1. Rakit dan jalankan `input-nama.asm`; catat hasilnya.
2. Ubah kapasitas penyangga 20 menjadi 5; jelaskan hasilnya.
3. Hapus baris `MOV BYTE PTR [SI], '$'`; jelaskan penyebab keluarannya.
4. Catat `SP` dan puncak stack sebelum serta sesudah `INT 21h` dengan `debug`.

Dikumpulkan: `tugas7.asm` (nama dan NIM, mode `03h`, fungsi `13h`, ditutup fungsi `4Ch`), tangkapan layar keluaran, tabel pengamatan `SP` dan puncak stack, serta jawaban langkah B.6 dan B.7.

Latihan

1. Jelaskan mengapa mekanisme interupsi lebih efisien daripada *polling*. Sertakan satu contoh perangkat pada komputer yang lebih baik diperiksa dengan interupsi.

2. Sebutkan tiga jenis interupsi pada prosesor 8086. Untuk setiap jenis, berikan satu contoh beserta nomor vektornya dan siapa yang memicunya.

3. Dengan memakai rumus alamat vektor$_n = n \times 4$, tentukan alamat entri vektor `0Ah`, `13h`, dan `21h`. Sebutkan pula alamat byte pertama (offset) dan byte ketiga (segmen) setiap entri.

4. Sebuah program berjalan dengan `SP` = `FF00h` ketika `INT 21h` dijalankan. Berapa `SP` sesudah ketiga penyimpanan selesai, nilai apa yang berada pada `SS:SP`, `SS:SP+2`, dan `SS:SP+4`, dan mengapa `RET` tidak dapat menggantikan `IRET`?

5. Tentukan warna huruf dan warna latar untuk atribut `1Eh`, `4Fh`, dan `07h`, dengan menyebut nilai bit 3–0, bit 6–4, dan bit 7.

Rangkuman

- Interupsi memaksa prosesor menghentikan sementara alur program untuk melayani suatu kejadian, lalu melanjutkannya kembali; mekanisme ini lebih hemat daripada *polling*.
- Berdasarkan sumbernya: interupsi perangkat keras (`INTR` dan `NMI`), interupsi perangkat lunak (`INT n`), dan pengecualian yang dibangkitkan prosesor, misalnya pembagian dengan nol.
- Tabel vektor menempati `0000h`–`03FFh` pada segmen nol; setiap vektor 4 byte (2 byte offset dan 2 byte segmen), sehingga alamat entri vektor ke-n adalah $n \times 4$.
- `INT n` menyimpan `FLAGS`, `CS`, lalu `IP` ke stack, mengosongkan `IF` dan `TF`, kemudian memuat `CS:IP` dari entri vektor. `IRET` mengembalikan ketiganya secara terbalik, setara `RET` ditambah `POPF`.
- BIOS melayani perangkat keras (`INT 10h` video, `INT 16h` keyboard, `INT 17h` printer), sedangkan `INT 21h` melayani keperluan tingkat program: masukan (`01h`, `0Ah`), keluaran (`02h`, `09h`), pengelolaan vektor (`25h`, `35h`), dan terminasi (`4Ch`).
- Fungsi `0Ah` memakai penyangga dengan tata letak dari DOS, dan isi vektor dapat diganti melalui fungsi `25h`; penangan baru wajib menyimpan register dan diakhiri `IRET`, sedangkan vektor lama wajib dipulihkan.

Cek Pemahaman

1. Setiap entri pada tabel vektor interupsi 8086 berukuran ...
   - 1.1 2 byte berisi offset penangan
   - 1.2 4 byte: 2 byte offset dan 2 byte segmen
   - 1.3 4 byte berisi nomor vektor
   - 1.4 8 byte: `IP`, `CS`, dan `FLAGS`

2. Alamat entri untuk vektor `10h` adalah ...
   - 2.1 `0010h`
   - 2.2 `0040h`
   - 2.3 `0100h`
   - 2.4 `0160h`

3. Urutan penyimpanan ke stack ketika `INT n` dijalankan adalah ...
   - 3.1 `IP`, `CS`, `FLAGS`
   - 3.2 `CS`, `IP`, `FLAGS`
   - 3.3 `FLAGS`, `CS`, `IP`
   - 3.4 `FLAGS`, `IP`, `CS`

Penutup

Sebelum pertemuan berikutnya

- Selesaikan Tugas 7: `tugas7.asm` (sapaan berwarna), tabel pengamatan `SP` dan puncak stack, serta tangkapan layar keluaran.
- Ulangi bagian yang belum dikuasai dengan menelusuri `input-nama.asm` satu instruksi demi satu pada *debugger*.
- Baca **Bab XV** — Debugging pada Level Instruksi.

Bacaan lanjutan: Bab XIV bagian Tabel Vektor Interupsi dan Memasang Penangan Interupsi Sendiri; daftar lengkap vektor interupsi beserta setiap fungsinya; pembahasan bentuk baku alamat jauh `DS:DX` pada Bab VI.

Terima kasih

Bahasa Pemrograman Tingkat Rendah (MWCA010) ▪ Semester Ganjil